

Rapport individuel — Itération 3

Alexis Briend (abd7852) — Projet Long SHDL

23 mai 2026

Contexte

Responsable de la **structure générale** et de l'**interprétation du SHDL** (SCRUM-23). Sprint 3 : mener la conversion CST → circuit de la combinatoire scalaire (sprint 2) jusqu'au **langage SHDL complet** — vecteurs, appels de sous-modules, séquentiel (mémoire) — puis assurer l'**intégration** de l'application (interface unifiée, build cross-OS).

Activités réalisées

- **Conversion vectorielle** (Bus, Subset, Names) : ExpressionBuilder vectorisé, LHS et paramètres vecteurs, peuplement de Module.Entrees/Sorties depuis la signature.
- **Appels de sous-modules multi-fichiers** (figure) : ModuleCallBuilder (ModuleCall → AppelModule), ModuleResolver + API Conversion.convert(main, library), déduplication des LHS au bit physique. **Modules internes \$** : *dispatch* vers BuiltinModules (dont \$BasculeD).
- **Conversion séquentielle** (:=) : MemoryAssignmentBuilder (désucrage maître-esclave), FreshNames (signaux internes), réécriture des auto-références (compteurs) ; spec passée en revue adversariale.
- **Structure de l'app** : intégration de notre interface (thème CSS clair/sombre) dans la branche de rendu, migration **JavaFX 25.0.3**, build **cross-OS** (Gradle/Makefile + GitHub Actions), correctifs IHM, suivi des bugs.

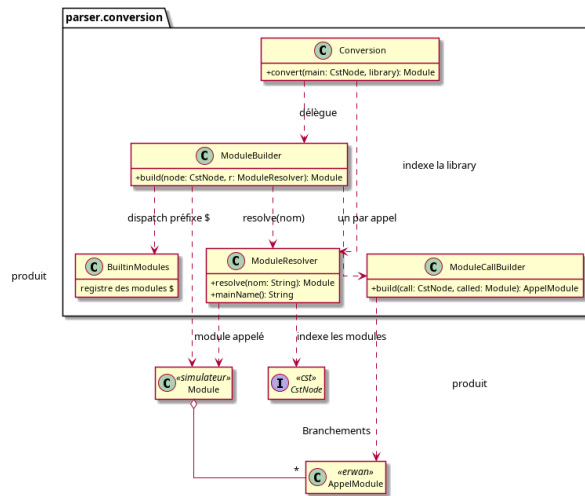


Figure — Conversion d'un appel de sous-module (parser.conversion).

Résultats

- **Conversion menée à terme** : 4 → 12 classes, 127 tests JUnit (14 classes de test) ; projet ≈ 389 tests (contre 193 au sprint 2). Pipeline SHDL complet : scalaire, vecteurs, appels, mémoire.
- **Livrable cross-OS** : *build* vert sur les trois OS, archive packagée (jar + JavaFX + lanceur + exemples). Bug amont du simulateur isolé par un test reproducteur (\$BasculeD).

Difficultés et réussites

Difficulté : le désucrage maître-esclave de la séquentielle (:=) et les auto-références, sécurisés par revue adversariale et tests bout-en-bout. Réussite : la **séparation stricte** parser / CST / conversion a tenu — la conversion a grandi (vecteurs, appels, mémoire) sans toucher au parser.

Manuel utilisateur

JDK 25. Pipeline : Conversion.convert(cst, library) (ConversionException avec offset), puis new FileSimulateur(m.Plan) ; modules internes via préfixe \$. Build : make build ou artefact CI par OS. Code sur feature/parser-111-table-driven et appearance-build.